

3rd Semester cse Branch

OPERATING SYSTEM (OS)

MOST EXPECTED QUESTION

**EDUCATION
PLUS+**



Operating System - Most Expected Questions

Bihar Engineering University | B.Tech CSE 3rd Semester

List and explain different types of Operating Systems.

1. Batch Operating System

- Executes jobs in batches without user interaction

2. Time-Sharing Operating System

- Multiple users share CPU time interactively

3. Multiprogramming Operating System

- Multiple programs kept in memory to maximize CPU usage

4. Multiprocessing Operating System

- Uses multiple CPUs for parallel processing

5. Distributed Operating System

- Manages multiple computers as a single system

6. Real-Time Operating System (RTOS)

- Provides quick response within strict time limits

Differentiate between Batch OS and Time-sharing OS.

Feature	Batch Operating System	Time-Sharing Operating System
Working	Jobs executed in batches without user interaction	Multiple users share CPU time simultaneously
User Interaction	No interaction during execution	Interactive system
Response Time	High (slow response)	Low (quick response)
CPU Utilization	Less efficient	More efficient
Examples	Early mainframe systems	Modern systems (Linux, Windows)

What is the role of an operating system as a resource manager?

- The OS manages all system resources (CPU, memory, I/O devices, files) and allocates them efficiently among competing processes.
- CPU Management: Schedules processes, handles context switching, and ensures fair CPU time distribution.
- Memory Management: Allocates/deallocates memory, handles virtual memory, and prevents processes from interfering with each other.
- I/O Device Management: Controls device access, manages device drivers, and handles I/O scheduling.
- File Management: Organizes files on disk, manages access permissions, and handles backup/recovery.

Discuss the evolution of Operating Systems from first generation to fourth generation with key features of each generation.

- First Generation (1945-1955): Vacuum tubes; no OS — programmers interacted directly with hardware via plugboards.
- Second Generation (1955-1965): Transistors; Batch systems — jobs were submitted on punch cards and processed sequentially by a monitor program.
- Third Generation (1965-1980): ICs; Multiprogramming and time-sharing — multiple jobs in memory, CPU switches between them; introduction of spooling.
- Fourth Generation (1980-present): VLSI/microprocessors; Personal computers with GUI-based OS — Windows, macOS, Linux; networking and distributed systems emerged.
- Each generation brought increased abstraction, better resource utilization, and improved user interaction from batch processing to interactive computing.

Define context switching. When does it occur?

Context switching is the process of saving the state (CPU registers, program counter, etc.) of a running process and loading the state of another process so that the CPU can switch execution.

When it Occurs:

- When a process is interrupted
- When a process moves to waiting state (I/O request)
- When a higher priority process arrives
- During time-sharing (time quantum expires)

What is the difference between preemptive and non-preemptive scheduling?

Feature	Preemptive Scheduling	Non-Preemptive Scheduling
CPU Control	CPU can be taken from a process	CPU not taken until process finishes
Response Time	Faster	Slower
Complexity	More complex	Simple
Examples	Round Robin, Priority (preemptive)	FCFS, SJF (non-preemptive)

What are threads? Discuss the multithreading models (Many-to-One, One-to-One, Many-to-Many).

Threads are lightweight units of a process that share the same memory space but execute independently.

Multithreading Models

1. Many-to-One

- Many user threads → one kernel thread
- Fast but blocking problem

2. One-to-One

- Each user thread → one kernel thread
- Better concurrency but more overhead

3. Many-to-Many

- Many user threads → many kernel threads
- Balanced approach (efficient + flexible)

Define race condition. Give an example.

A race condition occurs when two or more processes/threads access and modify shared data at the same time, and the final result depends on the order of execution.

Example:

Let a shared variable $x = 5$

- Process P1: $x = x + 1$
- Process P2: $x = x - 1$

If executed sequentially $\rightarrow$ final value = 5

But if executed simultaneously (race condition):

- Both read $x = 5$
- P1 writes 6, P2 writes 4

Final value may be 4 or 6 (incorrect)

Explain the Producer-Consumer problem.

It is a synchronization problem where producers generate data and consumers use that data, both sharing a common buffer.

Problem:

- If buffer is full → producer must wait
- If buffer is empty → consumer must wait
- Both should not access buffer simultaneously

Key Requirements:

- Mutual Exclusion → Only one process accesses buffer at a time
- Synchronization → Proper coordination between producer and consumer

What is the Readers-Writers problem? Explain briefly.

Readers–Writers Problem:

It is a synchronization problem where multiple processes share a common data resource.

- Readers → Only read data (can access simultaneously)
- Writers → Modify data (require exclusive access)

Key Idea:

- Multiple readers can read at the same time
- When a writer is writing → no other reader or writer can access the data

Issue:

- Need proper synchronization to avoid data inconsistency and starvation

Differentiate between deadlock prevention and deadlock avoidance.

Feature	Deadlock Prevention	Deadlock Avoidance
Approach	Break one of the deadlock conditions	Avoid unsafe states dynamically
Strategy	Restrictive rules (no hold & wait, etc.)	Check before allocation (safe state)
Resource Utilization	Low (underutilization possible)	Better utilization
Flexibility	Less flexible	More flexible
Example	No preemption, resource ordering	Banker’s Algorithm

Explain the circular wait condition with an example.

Circular wait is one of the necessary conditions for deadlock where a set of processes form a cycle, and each process is waiting for a resource held by the next process in the chain.

Explanation:

- P1 is waiting for a resource held by P2
- P2 is waiting for a resource held by P3
- ...
- Pn is waiting for a resource held by P1

This creates a circular chain, so none of the processes can proceed.

What is internal fragmentation? How does it differ from external fragmentation?

Feature	Internal Fragmentation	External Fragmentation
Location of Waste	Inside allocated block	Between free memory blocks
Cause	Fixed-size allocation (e.g., paging)	Variable-size allocation
Memory Usage	Space unused within block	Free space scattered
Solution	Reduce block size	Compaction or paging

Explain the concept of virtual memory.

Virtual memory is a memory management technique that allows a program to run without being fully loaded into RAM, by using disk storage as an extension.

Concept:

- Program is divided into pages
- Only required pages are loaded into RAM
- Remaining pages stay on disk and are loaded when needed (on demand)

Key Benefits:

- Run large programs with limited RAM
- Better memory utilization
- Supports multiprogramming

What is the dirty bit? Why is it important in page replacement?

- The dirty bit (modify bit) is a flag in the page table entry that indicates whether a page has been modified since it was loaded into memory.
- If dirty bit = 1 (modified): The page must be written back to disk before being replaced — its in-memory copy differs from the disk copy.
- If dirty bit = 0 (unmodified): The page can be simply overwritten (discarded) — the disk copy is already up to date.
- Importance: Reduces I/O operations — unmodified pages need not be written back, saving disk write time during page replacement.

What is Belady's Anomaly? Which algorithm suffers from it?

- Belady's Anomaly is the phenomenon where increasing the number of physical frames results in MORE page faults for certain reference strings.
- Only the FIFO page replacement algorithm suffers from Belady's Anomaly.
- Example: For reference string 1,2,3,4,1,2,5,1,2,3,4,5: FIFO with 3 frames gives 9 faults; with 4 frames gives 10 faults.
- Algorithms that are stack algorithms (LRU, Optimal) do NOT suffer from Belady's Anomaly — more frames always means fewer or equal faults.
- A stack algorithm guarantees that the set of pages in n frames is a subset of the pages in $n+1$ frames — FIFO violates this property.

Compare and explain the following page replacement algorithms: FIFO, LRU, Optimal, and Second Chance. Discuss Belady's Anomaly.

- FIFO: Replaces the oldest page (first in). Simple to implement but suffers from Belady's Anomaly — more frames can mean more faults.
- LRU (Least Recently Used): Replaces the page not used for the longest time. Good approximation of Optimal; no Belady's Anomaly; requires hardware support (counters or stack).
- Optimal: Replaces the page not used for the longest time in the FUTURE. Gives minimum page faults but is impossible to implement (requires future knowledge) — used only as a benchmark.
- Second Chance (Clock): Modified FIFO — gives a page a second chance using a reference bit; if bit=1, clear it and move to next; if bit=0, replace.
- Belady's Anomaly affects FIFO only. LRU and Optimal are stack algorithms — guaranteed that more frames = fewer faults.

Explain segmentation in detail. Compare segmentation with paging in terms of fragmentation, memory allocation, and hardware support.

Segmentation is a memory management technique where a program is divided into variable-size logical segments such as code, data, stack, functions, etc.

Key Concepts:

- Each segment has (base + limit)
 - Base → starting address
 - Limit → size of segment
- Logical address = (segment number + offset)
- Supports logical view of program

Feature	Segmentation	Paging
Fragmentation	External fragmentation	Internal fragmentation
Memory Allocation	Variable-size segments	Fixed-size pages
View	Logical (user view)	Physical (system view)
Address Format	Segment + offset	Page + offset
Hardware Support	Segment table (base & limit)	Page table

Consider the following page reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 with 3 frames. Calculate the number of page faults using FIFO, LRU, and Optimal algorithms.

Page reference string = 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Frames = 3

FIFO: Replace oldest page

Page Faults = 15

LRU: Replace least recently used page

Page Faults = 12

Optimal: Replace page not used for longest time in future

Page Faults = 9

Compare sequential and direct (random) file access methods.

Feature	Sequential Access	Direct (Random) Access
Access Method	Data accessed in order (one after another)	Data accessed directly at any position
Speed	Slower for large files	Faster access
Flexibility	Less flexible	More flexible
Use Case	Tape storage, logs	Disk storage, databases
Movement	Must go through previous records	Can jump directly to any record

Explain free-space management using bit vector and linked list.

Feature	Bit Vector (Bitmap)	Linked List
Concept	Uses bits to represent blocks (0 = free, 1 = used)	Free blocks are linked using pointers
Structure	Array of bits	Chain of free blocks
Memory Usage	Requires extra memory for bitmap	No extra structure (uses blocks themselves)
Searching	Faster to find free/contiguous blocks	Slower (need traversal)
Contiguous Allocation	Easy to find	Difficult to find
Implementation	Simple	Slightly complex
Efficiency	Good for large systems	Less efficient for large disks

What are the goals of I/O software?

- Device Independence: Programs should work regardless of the specific I/O device — uniform interface for all devices.
- Uniform Naming: Devices should be named in a consistent way (e.g., all files and devices accessed through path names in Unix).
- Error Handling: Errors should be handled at the lowest possible layer — hardware errors at driver level, software errors at OS level.
- Synchronous/Asynchronous Transfer: Support both blocking I/O (process waits) and non-blocking I/O (process continues while I/O proceeds).
- Buffering and Caching: Improve performance by buffering data in kernel space and caching frequently accessed data.

Explain the following disk scheduling algorithms : FCFS, SSTF, SCAN, and C-SCAN.

FCFS (First Come First Serve):

Requests are processed in the same order they arrive. It is simple but can result in high head movement.

SSTF (Shortest Seek Time First):

The disk head selects the request closest to its current position. It reduces movement but may cause starvation for distant requests.

SCAN (Elevator Algorithm):

The disk head moves in one direction, servicing requests along the way, and then reverses direction at the end. It provides better performance and fairness.

C-SCAN (Circular SCAN):

The disk head moves in one direction only. After reaching the end, it jumps back to the beginning and continues. It gives uniform waiting time to all requests.

Consider a disk queue with I/O requests on cylinders: 98, 183, 37, 122, 14, 124, 65, 67. The disk head is initially at cylinder 53. Calculate total head movement using FCFS, SSTF, SCAN, and C-SCAN algorithms.

Given:

Queue = 98, 183, 37, 122, 14, 124, 65, 67

Initial Head = 53

1. FCFS

Order:

$53 \rightarrow 98 \rightarrow 183 \rightarrow 37 \rightarrow 122 \rightarrow 14 \rightarrow 124 \rightarrow 65 \rightarrow 67$

Movement:

$= |53-98| + |98-183| + |183-37| + |37-122| + |122-14| + |14-124| + |124-65| + |65-67|$

$= 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2$

Total = 640

2. SSTF

Closest selection each time

Order:

$53 \rightarrow 65 \rightarrow 67 \rightarrow 37 \rightarrow 14 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183$

Movement:

$= 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59$

Total = 236

Consider a disk queue with I/O requests on cylinders: 98, 183, 37, 122, 14, 124, 65, 67. The disk head is initially at cylinder 53. Calculate total head movement using FCFS, SSTF, SCAN, and C-SCAN algorithms.

3. SCAN (Assume towards 0 first)

Order:

$53 \rightarrow 37 \rightarrow 14 \rightarrow 0 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183$

Movement:

$= 16 + 23 + 14 + 65 + 2 + 31 + 24 + 2 + 59$

Total = 236

4. C-SCAN (Assume towards higher direction)

Disk range: 0–199

Order:

$53 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183 \rightarrow 199 \rightarrow 0 \rightarrow 14 \rightarrow 37$

Movement:

$= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 23$

Total = 382

THANK YOU !!